© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_19

19. Native-Speed Machine Learning in Java: DJL, ONNX, and JNI

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand the current state of machine learning in Java along with considering practical aspects.

Introduction

Machine learning is no longer confined to Python ecosystems. As the demand for production-grade, scalable AI solutions grows, Java has emerged as a capable player in the machine learning landscape. Whether you're building backend microservices, real-time APIs, or enterprise-grade applications, Java offers powerful tools for integrating and deploying machine learning workflows.

This chapter explores the evolving landscape of machine learning in Java, offering a practical guide to integrating modern AI into Java-based applications. It begins by showcasing how developers can leverage pretrained models using Hugging Face APIs and continues with native approaches like the Deep Java Library (DJL) for model hosting and inference. It also covers ONNX runtime for cross-platform model portability, Java Native Interface (JNI) for integrating high-performance native ML runtimes, and DL4J for training and fine-tuning models entirely within the JVM. The chapter concludes with an overview of Apache OpenNLP, a robust library for performing core NLP tasks in Java.

By combining low-level control with high-level abstractions, Java now provides a full-stack experience for machine learning—from data preparation to inference—entirely within its familiar, type-safe ecosystem.

Using Hugging Face APIs in Java

Hugging Face is a leading platform in the machine learning ecosystem, particularly known for its extensive repository of state-of-the-art pretrained models in natural language processing (NLP), computer vision, and more. With an emphasis on accessibility and open-source collaboration, Hugging Face allows developers to easily integrate powerful models like GPT, BERT, T5, and Falcon into their applications via hosted inference APIs or locally deployed solutions.

Accessing Hugging Face Models via an API

To interact with Hugging Face models from a Java application, you can use their REST-based Inference API. This API provides a unified way to send inputs to a model and receive outputs such as generated text, classification results, or embeddings. Figure 19-1 shows the flow.

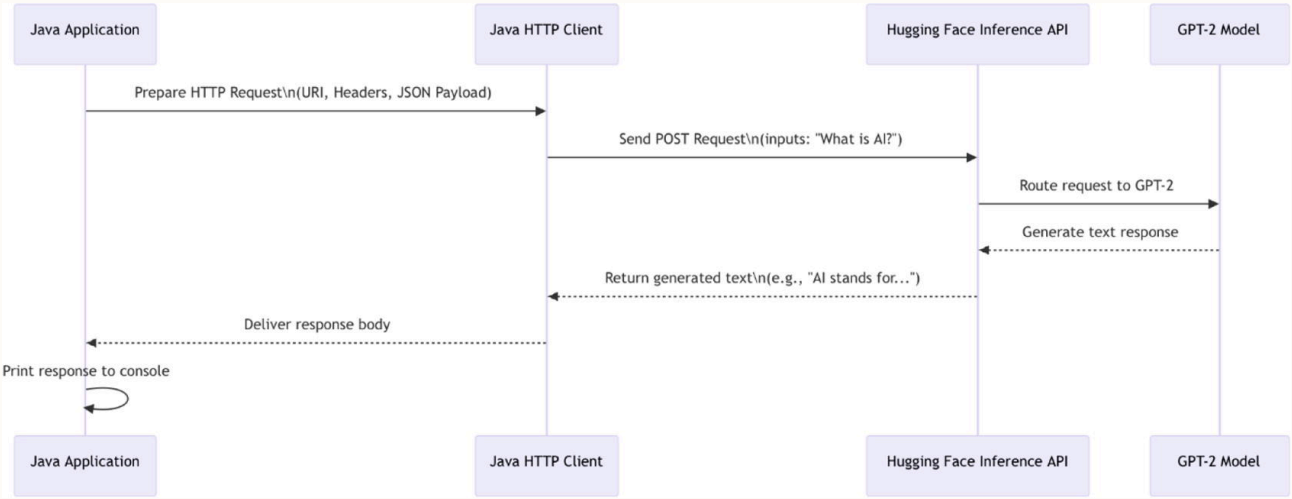


Figure 19-1 Request-response flow through different components

Here's an example of how you can invoke a text generation model (like GPT-2) directly from Java using native HTTP libraries:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
public class HuggingFaceExample {
    public static void main(String[] args) throws Exception {
        String apiKey = "YOUR_API_KEY";
        String input = "{\"inputs\": \"What is AI?\"}";
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("https://api-inference.huggingface.co/models/gpt2"))
                .header("Authorization", "Bearer " + apiKey)
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(input))
```

```
            .build();
        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandl
ers.ofString());
        System.out.println(response.body());
    }
}
```

This approach is lightweight and flexible, suitable for creating wrappers around different models and use cases—just as you would when integrating OpenAI APIs.

Creating Java Wrappers for Inference

To encapsulate and reuse API calls across your Java application, you can abstract the logic into service classes or utility methods. This pattern enhances testability, reusability, and configuration flexibility. Consider building a HuggingFaceClient class that handles authentication, request preparation, and response parsing.

These wrappers can be adapted to support different tasks such as:

- Text generation
- Text classification
- Translation
- Embedding extraction

You can extend your wrapper to dynamically choose models by passing the model ID as a parameter or configuring it via external properties. Figure 19-2 shows a sample block representation.

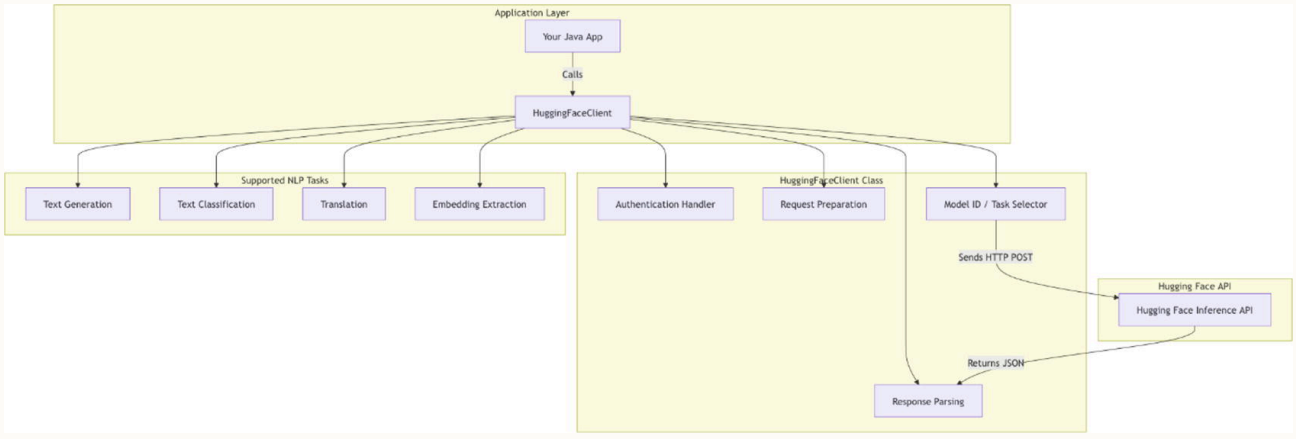


Figure 19-2 Inference wrappers for HuggingFace client

Integration via Frameworks

While direct API integration offers maximum control, Java frameworks like Spring AI and Quarkus with LangChain4j simplify the use of Hugging Face models through auto-configuration, dependency injection, and seamless service abstraction.

Spring AI

Spring AI provides first-class support for Hugging Face chat models and inference endpoints through minimal configuration. With the right properties in your `application.properties` or `application.yml` file, Spring Boot applications can use dependency injection to access a `HuggingfaceChatModel` bean, enabling text generation with minimal boilerplate.

Advantages:

- Declarative configuration
- Seamless integration into Spring Boot lifecycle
- Support for environment variable-based security
- Bill of materials (BOM) support for version consistency

Developers can also override configurations programmatically, allowing for custom authentication, dynamic endpoint selection, or runtime switching between models.

Quarkus + LangChain4j

For developers using Quarkus, the `langchain4j` extension supports Hugging Face integration with declarative configuration. Quarkus allows for enabling or disabling specific model types (chat, embedding, moderation), setting inference endpoint URLs, and adjusting generation parameters (like temperature, max tokens, top-k, top-p).

Advantages:

- Fast startup and low memory footprint for cloud-native deployments
- Build-time and runtime configurations
- Integrated model lifecycle management
- Support for both cloud-hosted and local Hugging Face models

Quarkus makes it easy to switch between OpenAI, Hugging Face, and other providers, which is ideal for applications that need flexibility or fallback options.

Hugging Face empowers Java developers to use LLMs through:

- **Direct REST API**: Customizable, lightweight, and framework-agnostic

- **Framework integration**: Simplified setup via Spring AI or Quarkus LangChain4j
- **Text Generation Inference (TGI)**: High-performance deployment options for large-scale use

Whether you're building simple utilities or enterprise-grade applications, Hugging Face's ecosystem and APIs provide the flexibility to innovate quickly with cutting-edge AI models.

Deep Java Library (DJL)

DJL is an open-source, high-level Java framework for deep learning that works seamlessly with multiple underlying engines. This is an initiative from Amazon. Designed with Java developers in mind, DJL feels just like any other Java library—easy to include in your projects, simple to use within your favorite IDE, and approachable even if you're new to machine learning.

With DJL, you don't need to be an expert in neural networks to get started. You can leverage your existing Java skills to build, train, and deploy models without leaving the JVM. DJL handles the details: it automatically detects and utilizes your hardware's CPU or GPU for optimal performance and lets you swap out deep learning engines (such as TensorFlow, PyTorch, or MXNet) at any time, giving you full flexibility to choose—or change—your backend.

DJL's intuitive, ergonomic API guides you through common deep learning tasks according to best practices. Whether you're prototyping a model, running inference in production, or embedding AI capabilities directly into a Java application, DJL provides a native, streamlined experience that keeps you focused on solving problems—not wrestling with toolchains. Figure 19-3 is the overall component architecture of the DJL framework.

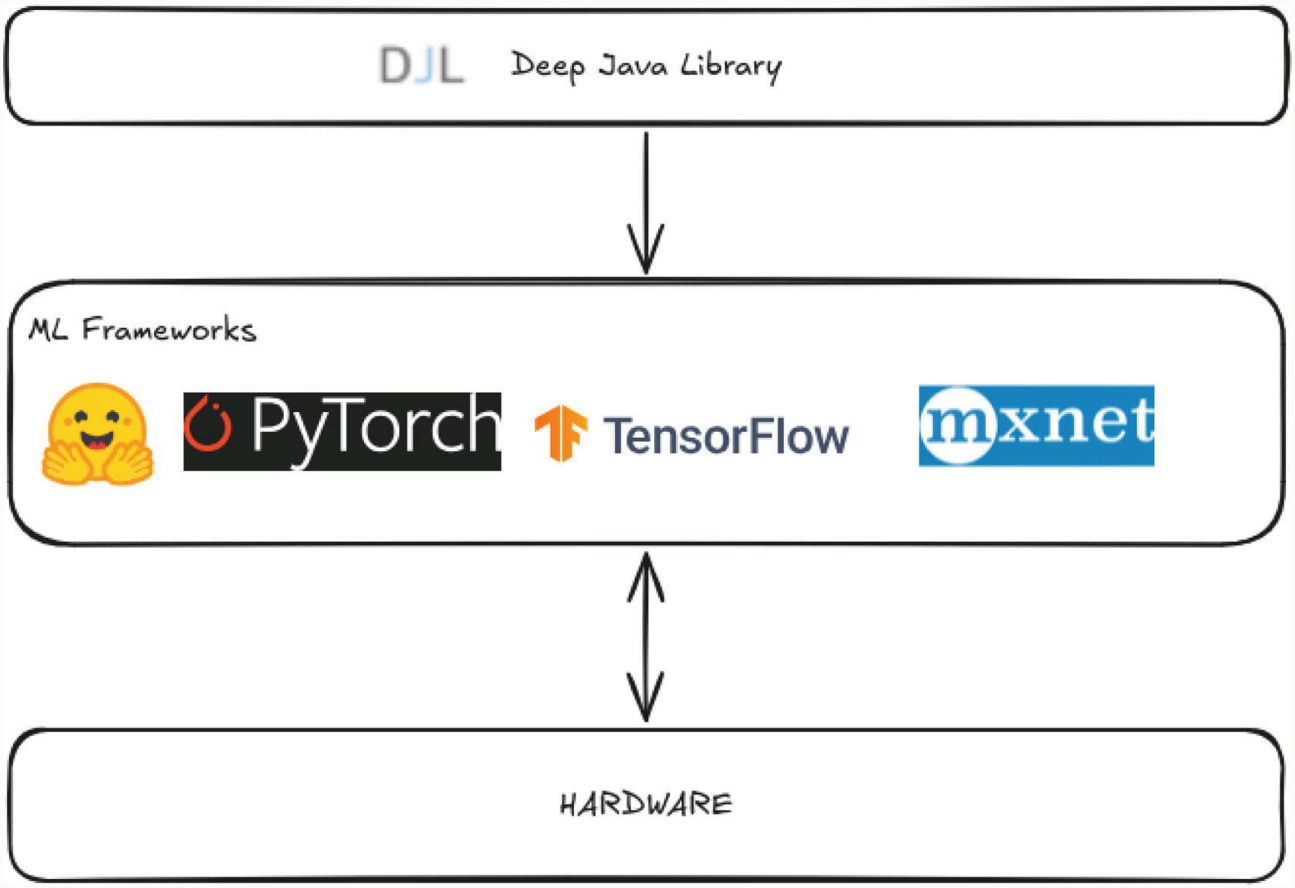


Figure 19-3 DJL architecture

Hosting and Inference Using DJL

Inference (or inferencing) in machine learning is the process of using a trained model to make predictions or decisions on new, unseen data. In this section, you will build a small application to help one of your customers who is into health-specific exercise suggestions. Based on the customers and users' physiological attributes, they suggest the number of iterations of exercises they will need to do. To do this, you will host an ONNX-based model which will help you predict these for your customers.

The model is available in the resources directory of the project for this chapter. You can also create this model by running the following code in a free Google Colab instance at https://colab.research.google.com/:

```
# -*- coding: utf-8 -*-
"""onnx_model_generation.ipynb
Automatically generated by Colab.
"""
!pip install skl2onnx onnxruntime
from sklearn.ensemble import RandomForestRegressor
from skl2onnx import to_onnx
from skl2onnx.common.data_types import FloatTensorType
# 1) Fit a RandomForestRegressor
rgr = RandomForestRegressor()
```

```
rgr.fit(X_train, y_train)
# 2) Define the ONNX input type explicitly
initial_type = [("float_input", FloatTensorType([None, X.shape[1]]))]
# 3) Convert to ONNX
onx = to_onnx(rgr, initial_types=initial_type)
with open("model.onnx", "wb") as f:
f.write(onx.SerializeToString())
print(pred_onx)
import numpy as np
from sklearn.datasets import load_linnerud
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor  # or RandomForestClassifier
from skl2onnx import to_onnx
from skl2onnx.common.data_types import FloatTensorType
import onnxruntime as rt
# 1) Load & split data
X, y = load_linnerud(return_X_y=True)
X = X.astype(np.float32)
y = y.astype(np.float32)  # for regressor; if classifier, y must be int labels
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
# 2) Train
model = RandomForestRegressor(random_state=0)  # replace with RandomForestClassifi
er() if doing classification
model.fit(X_train, y_train)
# 3) Convert to ONNX
initial_type = [("input", FloatTensorType([None, X.shape[1]]))]
onnx_model = to_onnx(model, initial_types=initial_type)
with open("model.onnx", "wb") as f:
f.write(onnx_model.SerializeToString())
# 4) Inference with ONNX Runtime
sess = rt.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])
input_name  = sess.get_inputs()[0].name
output_name = sess.get_outputs()[0].name
# For regressor this is your continuous output;
# for classifier it'll be predicted label(s), and you can also fetch probabilities
y_pred_onx = sess.run([output_name], {input_name: X_test})[0]
print("ONNX prediction shape:", y_pred_onx.shape)
print("First 5 ONNX predictions:\n", y_pred_onx[:5])
X_test
```

I will not focus too much on the model generation part since as developers we would probably use one created by the machine learning team. Once you have run the code using a GPU runtime in Google Colab, store this file in the resource directory.

This model has been created on top of the Linnerud dataset.

Understanding the Linnerud Dataset

The Linnerud dataset is a small, classic multi-output regression dataset commonly used in educational and prototyping contexts. It provides a simple yet effective example of how physiological metrics can be used to predict physical performance outcomes.

Originally collected in a fitness study, the dataset contains observations from 20 middle-aged men, measuring their physical characteristics alongside their performance on three types of exercises. This dataset is particularly useful for demonstrating supervised learning tasks where multiple continuous outputs are predicted simultaneously.

Dataset Structure

The dataset is composed of two CSV files representing features (inputs) and targets (outputs).

Physiological Data: Input Features (X)

These are the independent variables used as inputs for model training:

- **Weight**: Body weight (in kilograms)
- **Waist**: Waist circumference (in centimeters)
- **Pulse**: Resting pulse rate (beats per minute)

Exercise Data: Target Variables (Y)

These are the dependent variables or labels to be predicted:

- **Chins**: Number of chin-ups completed
- **Situps**: Number of sit-ups completed
- **Jumps**: Number of jumps (in centimeters)

Each row in the dataset corresponds to one individual and includes a set of physiological inputs and matching exercise outputs.

Use in Machine Learning

The Linnerud dataset is ideal for training and evaluating multi-output regression models, where the goal is to predict multiple continuous values simultaneously from a single in-

put vector.

In our use case, we can train a regression model that takes a user's physiological profile (weight, waist, pulse) and predicts their expected performance in chins, situps, and jumps. Once trained, this model can be exported to the ONNX format, allowing it to be deployed across various platforms—including Java applications using ONNX Runtime.

Example Workflow:

1. **Preprocessing**: Normalize or scale the data as needed.
2. **Model training**: Use a regression algorithm (e.g., linear regression, MLP regressor) to map inputs to multiple outputs.
3. **ONNX export**: Save the trained model in ONNX format for cross-platform inference.
4. **Integration**: Load the ONNX model in a Java application and use it to predict exercise capacity based on user input.

Why Use This Toy Dataset?

While the Linnerud dataset is limited in size, it is excellent for:

- **Educational purposes**: Understanding the mechanics of multi-output regression
- **Proof of concept**: Testing model training and ONNX export pipelines
- **Integration testing**: Validating end-to-end inference in Java using small, manageable input/output dimensions

Building an ONNX Regression Model with the Linnerud Dataset

To predict user performance based on physiological metrics, you've selected the Linnerud dataset—a multi-output regression dataset ideal for small-scale model prototyping. The dataset's structure lends itself well to training models that can predict multiple targets simultaneously (e.g., predicting chin-ups, sit-ups, and jumps from weight, waist, and pulse).

Step-by-Step: Training and Exporting the Model

The model training and conversion process is implemented in Python using scikit-learn and skl2onnx. Here's a breakdown of each key step:

Step 1: Load and Prepare the Dataset

The dataset is loaded using sklearn.datasets.load_linnerud(), which returns physiological data as features (X) and exercise performance metrics as labels (y). Both are cast to float32 to ensure compatibility with ONNX.

```
X, y = load_linnerud(return_X_y=True)
X = X.astype(np.float32)
y = y.astype(np.float32)
```

Step 2: Train a Regression Model

A RandomForestRegressor is trained using the Linnerud physiological data to predict the exercise outcomes. This model is capable of handling multi-output regression without modification.

```
model = RandomForestRegressor(random_state=0)
model.fit(X_train, y_train)
```

Step 3: Convert the Model to ONNX Format

The trained model is converted to ONNX using skl2onnx.to_onnx(). The input type is explicitly defined using FloatTensorType, with the shape based on the feature count (X.shape[1]).

```
initial_type = [("input", FloatTensorType([None, X.shape[1]]))]
onnx_model = to_onnx(model, initial_types=initial_type)
with open("model.onnx", "wb") as f:
  f.write(onnx_model.SerializeToString())
```

This produces a model.onnx file, which is portable and can be loaded in any ONNX-compatible runtime—including ONNX Runtime for Java.

Step 4: Test Inference in Python

To verify that the model behaves as expected, inference is run using onnxruntime in Python. This helps validate that the ONNX export was successful and gives a reference prediction to compare against during Java-side integration.

```
sess = rt.InferenceSession("model.onnx", providers=["CPUExecutionProvider"])
input_name  = sess.get_inputs()[0].name
```

```
output_name = sess.get_outputs()[0].name
y_pred_onx = sess.run([output_name], {input_name: X_test})[0]
```

Why This Workflow Works

This approach allows you to:

- Leverage Python's robust ML ecosystem for model training.
- Export to ONNX, a portable format.
- Use ONNX Runtime in Java to perform inference—eliminating the need for a Python runtime in production.
- Apply predictions in a Java-based system (e.g., Spring Boot API) using user inputs like weight, waist, and pulse.

Now that you have the ONNX format based model available to use, you can host it using Spring Boot. Let's build a starter template using Spring Initializr, as shown in Figure 19-4.

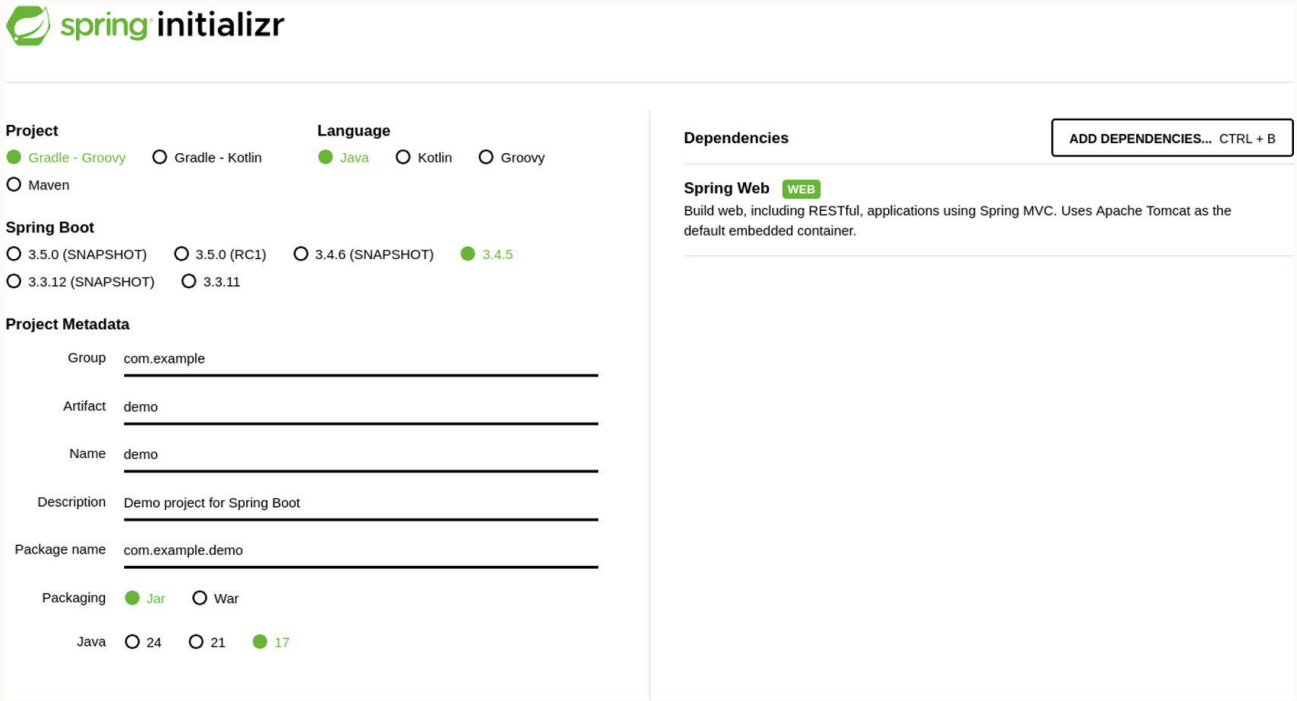


Figure 19-4 Spring Initializr for ONNX model hosting

First, we will update the build.gradle file with our onnxruntime-engine library dependency from DJL as shown here:

File: ./build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.4.5'
```

```
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}
repositories {
    mavenCentral()
}
ext {
    set('aiDjlVersion', "0.33.0")
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'ai.djl.onnxruntime:onnxruntime-engine'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
dependencyManagement {
    imports {
        mavenBom "ai.djl:bom:${aiDjlVersion}"
    }
}
tasks.named('test') {
    useJUnitPlatform()
}
```

Let's now create a few model POJOs for the request and response objects as follows:

File: ./src/main/java/com/example/demo/model/PhysiologicalInput.java

```
package com.example.demo.model;
public class PhysiologicalInput {
    public float weight;
    public float waist;
    public float pulse;
    public PhysiologicalInput(float weight, float waist, float pulse) {
        this.weight = weight;
        this.waist = waist;
        this.pulse = pulse;
    }
}
```

File: ./src/main/java/com/example/demo/model/PredictionResponse.java

```
package com.example.demo.model;
public record PredictionResponse(int chins, int situps, int jumps) {}
```

To perform inference on an ONNX-based regression model from a Java application, you will use DJL—a high-level framework designed to bring deep learning to the JVM. In this setup, the input to the model is represented by a custom class called PhysiologicalInput, which captures three physiological features: weight, waist, and pulse. These features are passed into a custom Translator class named PhysiologyTranslator, which implements NoBatchifyTranslator<PhysiologicalInput, float[]>. Inside the processInput() method, the physiological values are assembled into a float array and converted into a DJL-compatible NDArray with a shape of (1, 3) using the following line:

```
NDArray array = ctx.getNDManager().create(data, new Shape(1, 3));
```

This preprocessing step ensures that the model receives input in the same format it was trained on. Once the model produces an output, the processOutput() method extracts the result as a simple float array.

```
float[] data = list.get(0).toFloatArray();
```

This output corresponds to the predicted number of chins, situps, and jumps, making it a multi-output regression task.

File: ./src/main/java/com/example/demo/ml/PhysiologyTranslator.java

```
package com.example.demo.ml;
import ai.djl.ndarray.NDArray;
import ai.djl.ndarray.NDList;
import ai.djl.ndarray.types.Shape;
import ai.djl.translate.NoBatchifyTranslator;
import ai.djl.translate.TranslatorContext;
import com.example.demo.model.PhysiologicalInput;
public class PhysiologyTranslator
        implements NoBatchifyTranslator<PhysiologicalInput, float[]> {
    public PhysiologyTranslator() {}
    @Override
    public NDList processInput(TranslatorContext ctx, PhysiologicalInput input) {
        float[] data = {input.waist, input.weight, input.pulse};
        NDArray array = ctx.getNDManager().create(data, new Shape(1, 3));
```

```
        return new NDList(array);
    }
    @Override
    public float[] processOutput(TranslatorContext ctx, NDList list) {
        float[] data = list.get(0).toFloatArray();
        return data;
    }
}
```

The model itself is loaded using DJL's Criteria API, which specifies the input/output types, the ONNX model location, and the use of the OnnxRuntime engine:

```
Criteria.builder()
        .setTypes(PhysiologicalInput.class, float[].class)
        .optModelUrls(modelLocation)
        .optTranslator(new PhysiologyTranslator())
        .optEngine("OnnxRuntime")
        .build();
```

File: ./src/main/java/com/example/demo/configuration/ModelConfiguration.java

```
package com.example.demo.configuration;
import ai.djl.inference.Predictor;
import ai.djl.repository.zoo.Criteria;
import ai.djl.repository.zoo.ZooModel;
import com.example.demo.ml.PhysiologyTranslator;
import com.example.demo.model.PhysiologicalInput;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.util.function.Supplier;
@Configuration
public class ModelConfiguration {
    @Bean
    public Criteria<PhysiologicalInput, float[]> criteria() {
        String modelLocation = Thread.currentThread()
                .getContextClassLoader()
                .getResource("model.onnx")
                .getPath();
        return Criteria.builder()
                .setTypes(PhysiologicalInput.class, float[].class)
                .optModelUrls(modelLocation)
                .optTranslator(new PhysiologyTranslator())
                .optEngine("OnnxRuntime")
```

```
                .build();
    }
    @Bean
    public ZooModel<PhysiologicalInput, float[]> model(
            @Qualifier("criteria") Criteria<PhysiologicalInput, float[]> criteria)
            throws Exception {
        return criteria.loadModel();
    }
    @Bean
    public Supplier<Predictor<PhysiologicalInput, float[]>>
    predictorProvider(ZooModel<PhysiologicalInput, float[]> model) {
        return model::newPredictor;
    }
}
```

This model configuration is defined as a Spring @Bean and injected into the application context, making it easy to manage lifecycle and dependencies.

Finally, inference is exposed through a REST API in the ExerciseMetricInferenceController class. The controller defines a POST endpoint at /inference, which accepts JSON input, invokes the model using a Predictor, and maps the raw float outputs into a structured PredictionResponse.

```
float[] response = predictor.predict(physiologicalInput);
return new PredictionResponse((int) response[0], (int) response[1], (int) response[2]);
```

File: ./src/main/java/com/example/demo/controller/ExerciseMetricInferenceController.java

```
package com.example.demo.controller;
import ai.djl.inference.Predictor;
import ai.djl.translate.TranslateException;
import com.example.demo.model.PredictionResponse;
import com.example.demo.model.PhysiologicalInput;
import jakarta.annotation.Resource;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import java.util.function.Supplier;
@RestController
public class ExerciseMetricInferenceController {
    @Resource
    private Supplier<Predictor<PhysiologicalInput, float[]>> predictorSupplier;
    @PostMapping("/inference")
```

```
    PredictionResponse detectExerciseMetric(@RequestBody PhysiologicalInput physiol
ogicalInput)
            throws TranslateException {
        try (var p = predictorSupplier.get()) {
            float[] response = p.predict(physiologicalInput);
            return new PredictionResponse(((int) response[0]), (int) response[1], (int) resp
onse[2]);
        }
    }
}
```

This end-to-end integration demonstrates how DJL simplifies loading ONNX models, handling input/output translation, and deploying inference in production-ready Spring Boot applications. It mirrors DJL's well-known Iris example but applies it to a real-world regression problem using a custom-trained model and domain-specific data.

Let's boot up our application and see the result in action by making a curl request. You will set up some DJL library-specific logs on boot-up. These are only informational at this stage.

```
2025-05-21T10:39:35.490Z  WARN 2223 --- [demo] [main] ai.djl.repository.SimpleRe
pository      : Simple repository pointing to a non-archive file.
2025-05-21T10:39:35.756Z  WARN 2223 --- [demo] [main] ai.djl.onnxruntime.engine.
OrtEngine     : CUDA is not supported OnnxRuntime engine: Error code - ORT_EP_FAIL -
message: Failed to find CUDA shared provider
2025-05-21T10:39:35.767Z  INFO 2223 --- [demo] [main] ai.djl.onnxruntime.engine.O
rtModel       : Onnx extension not found in classpath.
curl -X POST http://localhost:8080/inference   -H "Content-Type: application/json"   -
d '{"weight": 70.5, "waist": 32.0, "pulse": 72.0}'
```

Output:

```
{"chins":190,"situps":36,"jumps":53}
```

So, for a customer who has physiological attributes similar to the previous, they need to perform 100 chin-ups, 36 sit-ups, and 53 jumps. Our customers are very happy after this API integration to their health app, which helps them decide the number of exercises of each type. Figure 19-5 shows the overall request-response control flow.

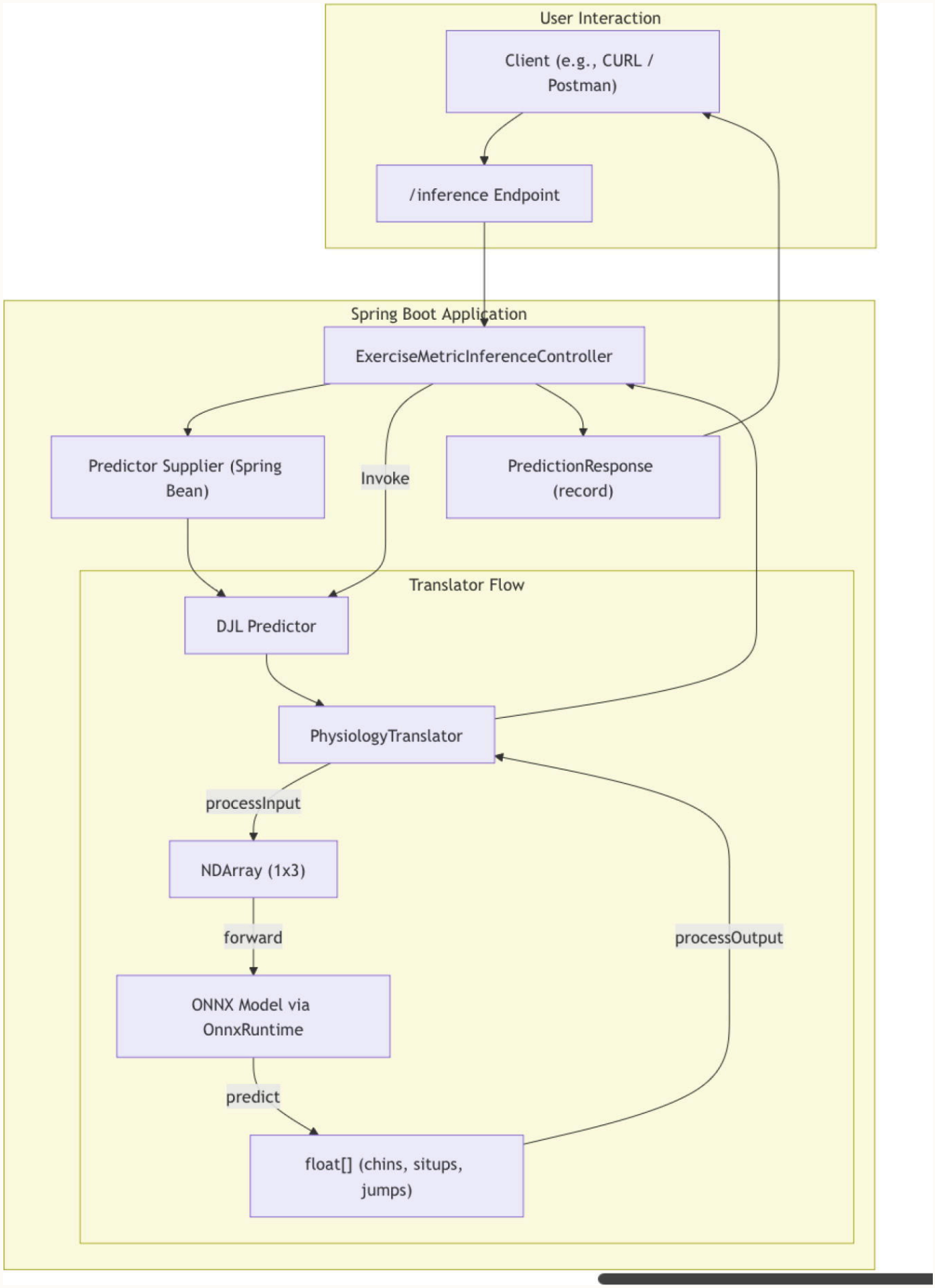


Figure 19-5 Onnx regression inference flow in Spring

Integrating LLMs Using Java Native Interface

Java Native Interface (JNI) is a standard mechanism provided by the Java platform that enables Java code to interact with native applications and libraries written in languages like C, C++, or even Python (indirectly via bridges). It serves two primary purposes:

- **Calling native code from Java**: Enables execution of native functions (e.g., C/C++ libraries) from within a Java application.
- **Embedding the JVM in native applications**: Allows non-Java applications to embed and use the Java Virtual Machine (JVM).

JNI is particularly valuable in scenarios where high performance is essential, such as integrating with native machine learning inference engines or LLMs deployed in C++ or Python environments.

Why Use JNI for LLM Integration?

While Java is portable and secure, it may not always provide the lowest-level performance for compute-intensive operations. Many state-of-the-art LLMs are implemented in Python (via frameworks like PyTorch) or C++ (e.g., GGML-based runtimes). JNI allows developers to bridge the gap and directly call into these native implementations, resulting in:

- Improved inference speed
- Reduced overhead from inter-process communication
- Fine-grained control over memory and model execution

However, JNI comes with trade-offs—most notably, the added complexity of memory management and the loss of some Java safety guarantees.

Sample JNI Integration with a Native LLM

The following is a simple example showing how a Java application might use JNI to invoke a function implemented in a native shared library (e.g., a C++ backend for an LLM):

```
public class JNILlmIntegrationExample {
    static {
        System.loadLibrary("nativeLLM"); // Loads the native shared library (e.g., libnativeLLM.so or nativeLLM.dll)
    }
    // Native method declaration
    public native String runLLM(String input);
    public static void main(String[] args) {
        JNILlmIntegrationExample integration = new JNILlmIntegrationExample();
        String output = integration.runLLM("What is artificial intelligence?");
        System.out.println("LLM Response: " + output);
    }
```

```
}
```

Behind the Scenes

To make the previous example work, the following steps are generally required:

1. Generate the header file from the Java class using javac and javah (or javac -h).
2. Implement the native method in C or C++, compiling it into a shared library.
3. Load the library at runtime with System.loadLibrary.

The native method could then call into an optimized C++ inference engine (e.g., llama.cpp) or wrap Python-based models using embedding tools like pybind11.

When to Use JNI for LLMs

JNI is most appropriate when:

- You need ultra-low latency inference, such as on-device or edge computing.
- You're working with an existing native codebase and want to avoid rewriting logic in Java.
- Memory usage and threading must be tightly controlled, such as with large GPU/CPU-bound models.

Considerations

Note the following:

- Error handling and debugging can be more complex across the Java-native boundary.
- Platform-specific dependencies require careful management for cross-platform support.
- Garbage collection coordination between Java and native code is essential to avoid leaks or crashes.

By using JNI, Java applications can tap directly into the power of native LLMs while still leveraging Java's ecosystem for orchestration, interfaces, and integration. It's a powerful, albeit advanced, technique for performance-critical AI workloads.

Running Inference in Java with ONNX Runtime

The Open Neural Network Exchange (ONNX) format is a widely supported standard for representing machine learning models trained in frameworks like PyTorch, TensorFlow, and scikit-learn. It allows developers to export models into a framework-agnostic format and run them across multiple platforms and environments.

To leverage these models in Java applications, the ONNX Runtime (ORT) provides official Java bindings. This enables developers to execute high-performance inference directly on the JVM without relying on Python or other external runtimes.

Why Use ONNX Runtime in Java?

Using ONNX Runtime in Java offers several benefits:

- **Cross-framework support**: Run models trained in different ML frameworks using a consistent Java API.
- **Hardware acceleration**: Supports CPU and GPU backends, making it suitable for scalable deployments.
- **Minimal dependencies**: No Python runtime required—everything runs natively on the JVM.

Getting Started with ONNX Runtime for Java

To use ONNX Runtime in your Java project, include the ONNX Runtime Java bindings in your build system (e.g., via Maven or Gradle). Then, you can load a pretrained model and run inference as follows.

Here's a basic example using a dummy input tensor with a different setup than the original example:

```
import ai.onnxruntime.OnnxTensor;
import ai.onnxruntime.OrtEnvironment;
import ai.onnxruntime.OrtSession;
import ai.onnxruntime.OrtException;
import java.nio.FloatBuffer;
import java.util.Collections;
import java.util.Map;
public class ONNXInferenceDemo {
    public static void main(String[] args) throws OrtException {
        // Initialize ONNX environment and session with a model
       OrtEnvironment environment = OrtEnvironment.getEnvironment();
        OrtSession session = environment.createSession("classifier_model.onnx");
        // Create dummy input (e.g., a vector of 4 float values)
```

```
        float[] input = {1.0f, 0.5f, -1.2f, 3.3f};
         FloatBuffer buffer = FloatBuffer.wrap(input);
         long[] shape = {1, 4}; // Shape must match model input
        OnnxTensor inputTensor = OnnxTensor.createTensor(environment, buffer, shape);
         // Perform inference
        Map<String, OnnxTensor> inputs = Collections.singletonMap("input_name", inputT
ensor);
         OrtSession.Result output = session.run(inputs);
         // Extract output tensor (e.g., classification scores)
        float[][] result = (float[][]) output.get(0).getValue();
         for (float score : result[0]) {
            System.out.println("Score: " + score);
         }
      }
}
```

NoteReplace input_name with the actual input name from your ONNX model. This can be inspected using tools like Netron or programmatically via the session metadata.

Key Concepts

These are the key concepts:

- **OrtEnvironment**: The entry point to ONNX Runtime, managing global resources
- **OrtSession**: Represents a loaded ONNX model. You use this to run inference
- **OnnxTensor**: A wrapper around Java-native types (float, int, etc.) used for feeding data into the model

Use Cases

ONNX Runtime is well-suited for:

- **Java-based backend services** requiring ML predictions.
- **Microservices** that need to stay within the JVM ecosystem.
- **Desktop or Android applications** looking to embed efficient ML inference.

ONNX Runtime for Java bridges the gap between production-grade ML models and enterprise Java applications. By using the ONNX format and the ORT Java API, developers can deploy models without leaving the Java ecosystem—eliminating the overhead of external processes and improving performance in ML-enabled applications.

Model Training and Fine-Tuning Using DL4J

Eclipse Deeplearning4j (DL4J) is a comprehensive deep learning framework built specifically for the Java Virtual Machine (JVM). Unlike many popular ML frameworks that focus on Python, DL4J allows developers to train and deploy deep learning models entirely in Java, making it ideal for JVM-based systems, microservices, and enterprise-grade applications.

DL4J supports a wide variety of use cases—from training neural networks from scratch to importing and fine-tuning models built in other frameworks like PyTorch, TensorFlow, and Keras. Its design also bridges the Java and Python ecosystems through native interop features and Python execution bindings.

Key Components of the DL4J Suite

DL4J is more than just a training API—it's a full ecosystem composed of several modular libraries:

- **Deeplearning4j core**: High-level APIs for defining and training neural networks using MultiLayerNetwork or ComputationGraph.
- **SameDiff**: A low-level graph-based API, similar in spirit to TensorFlow or PyTorch, that supports dynamic graph definition and automatic differentiation.
- **ND4J**: A powerful Numpy-like library for Java, used for tensor computations and mathematical operations.
- **LibND4J**: A performance-optimized C++ backend for executing math operations across CPU, GPU, or specialized hardware.
- **DataVec**: A data transformation library that converts raw data into tensors, suitable for machine learning models.
- **Python4J**: A bridge to execute Python code from within Java, useful for hybrid workflows.
- **Apache Spark Integration**: Enables distributed model training on Spark clusters for handling large-scale datasets.

Training and Fine-Tuning Models

DL4J enables you to both train models from scratch and fine-tune pretrained ones. You can build models using either the high-level API (MultiLayerConfiguration, ComputationGraphConfiguration) or the lower-level SameDiff API.

Here are some example use cases:

- **Train a CNN on image data**: Using Tiny ImageNet or MNIST datasets.

- **Fine-tune a pretrained Keras model**: Import it into DL4J using KerasModelImport and retrain with your own dataset.
- **Custom NLP models**: Use word embeddings and LSTMs for classification tasks such as patent classification.

NoteFor running inference in parallel (e.g., batch serving), DL4J also supports parallelized inference pipelines.

Why Use DL4J?

DL4J is a strong choice when:

- You're operating in a Java-first environment (e.g., Spring Boot, JVM microservices).
- You want to avoid switching to Python for ML tasks.
- You need to integrate with Spark or other JVM-based big data tools.
- You're building for IoT, Android, or embedded systems where Java is more prevalent.

Open Source and Community

DL4J is open-source under the Apache 2.0 license, governed by the Eclipse Foundation. Contributions are welcome, and developers can explore examples, file issues, or submit pull requests via the official GitHub repositories.

Deeplearning4j enables full-cycle machine learning in Java—from preprocessing with DataVec to training with ND4J and SameDiff, and finally deploying models using ONNX or TensorFlow runtime interop. It's a JVM-native option that combines performance, flexibility, and production readiness for modern AI applications.

Natural Language Processing Using Apache OpenNLP

Apache OpenNLP is a pure-Java library that provides a range of ready-made tools for common NLP tasks. It's lightweight, engine-agnostic, and easy to drop into any Java application. Here's how it can help you add language understanding to your projects:

- **Sentence detection:** Split raw text into individual sentences.
- **Tokenization:** Break sentences into words or tokens.
- **Part-of-speech tagging:** Label each token with its grammatical role (noun, verb, adjective, etc.).
- **Named-entity recognition (NER):** Identify proper names such as people, places, dates, and organizations.

- **Chunking and parsing:** Extract phrase structure (noun phrases, verb phrases) or full parse trees.
- **Language detection:** Guess the language of a given text.

Getting Started

Here are the steps to get started:

1. Add the dependency to your build tool (Maven example):

```
<dependency>
    <groupId>org.apache.opennlp</groupId>
    <artifactId>opennlp-tools</artifactId>
    <version>2.5.4</version>
</dependency>
```

2. Load a pretrained model (models available on the Apache website):

```
try (InputStream modelIn = new FileInputStream("en-token.bin")) {
    TokenizerModel model = new TokenizerModel(modelIn);
    TokenizerME tokenizer = new TokenizerME(model);
    String[] tokens = tokenizer.tokenize("Hello, OpenNLP makes NLP in Java simpl
e!");
}
```

3. Run a task, for example, named-entity recognition:

```
try (InputStream nerModelIn = new FileInputStream("en-ner-person.bin")) {
    TokenNameFinderModel nerModel = new TokenNameFinderModel(nerModelI
n);
    NameFinderME nameFinder = new NameFinderME(nerModel);
    Span[] names = nameFinder.find(tokens);
    for (Span name : names) {
        System.out.println("Entity: " +
                String.join(" ", Arrays.copyOfRange(tokens, name.getStart(), name.getE
nd())));
    }
}
```

Why Use OpenNLP?

These are the reasons to use it:

- **No external dependencies**: Everything runs on the JVM.
- **Speed**: Optimized for production workloads.
- **Flexibility**: Swap in your own custom-trained models as your use case grows.
- **Simplicity:** A consistent, intuitive API surface for all tasks.

With Apache OpenNLP, you can rapidly prototype and deploy NLP pipelines—whether you need to analyze customer feedback, extract information from documents, or build chatbots—without leaving the Java ecosystem.

Conclusion

Java's capabilities in machine learning have matured significantly, enabling developers to build intelligent applications without switching ecosystems. Java provides high-level abstraction, excellent technical representation of concepts, and low-level control of details. As we've seen, there are multiple paths to bringing AI into the Java world—from consuming APIs hosted on Hugging Face to building and deploying ONNX models with DJL. Tools like the ONNX Runtime for Java and DL4J offer flexibility for both inference and training, while JNI bridges the gap to native runtimes when performance and custom integrations matter. Meanwhile, Apache OpenNLP showcases how natural language processing tasks can be seamlessly embedded into Java applications without introducing external dependencies.

This chapter demonstrates that Java is no longer just an enterprise or backend language—it is also a viable, powerful platform for modern machine learning. Whether you're consuming, serving, or training models, the Java ecosystem now supports every major phase of the ML lifecycle. With robust libraries, growing community support, and production-grade deployment options, Java is fully equipped to support AI-driven innovation across domains.

And now for reviewing the architecture of tomorrow in the LLM ecosystem in the next chapter.

Previous chapter
‹ 18. Can You See Me Now? Observability for LLM

Next chapter
20. Architectures of Tomorrow: From Monoliths to Modular ›

Pipelines

Minds